Programación por bloques

Nombre: _____ Fecha: _____

Qué vas a aprender

- Entender qué es un algoritmo y representarlo con diagramas de flujo
- Conocer el entorno de programación por bloques y sus elementos: objetos, eventos y scripts
- Usar secuencias, bucles, condicionales, variables y operadores
- Diseñar, probar y depurar programas sencillos, como juegos y animaciones

Para pensar

Un videojuego sencillo puede tener cientos de órdenes. ¿Cómo sabe el ordenador cuándo debe ejecutar cada una?

Explicación

1.1 Algoritmos y programas

Un algoritmo es una secuencia **ordenada y finita** de instrucciones claras para resolver un problema o realizar una tarea. Una receta de cocina, las instrucciones de montaje de un mueble o los pasos para resolver una ecuación son algoritmos. Un programa es un algoritmo escrito en un **lenguaje de programación** que el ordenador puede ejecutar. Los ordenadores no piensan: hacen **exactamente** lo que se les ordena, en el orden indicado, por eso las instrucciones deben ser precisas y sin ambigüedades. Programar es descomponer un problema grande en pasos pequeños, una habilidad llamada **pensamiento computacional**, útil mucho más allá de la informática.

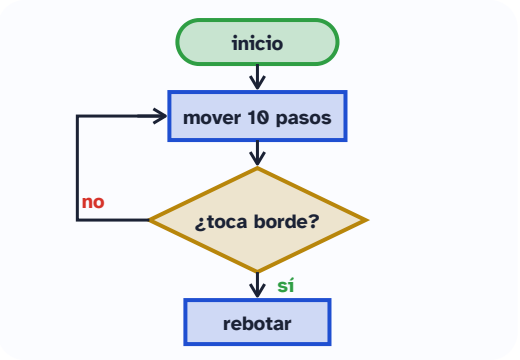
Un buen algoritmo cumple varias propiedades: es **preciso** (cada paso se entiende de una sola manera), **finito** (termina en algún momento), **ordenado** y tiene unas **entradas** (datos de partida) y unas **salidas** (resultado). Cuando el algoritmo se escribe en un lenguaje que entiende el ordenador, se convierte en un **programa**. Antes de programar se puede escribir en **pseudocódigo**: frases sencillas, numeradas, a medio camino entre el lenguaje natural y el de programación.



1.2 Diagramas de flujo

Antes de programar conviene representar el algoritmo de forma gráfica con un diagrama de flujo. Usa símbolos normalizados unidos por flechas que indican el orden: el **óvalo** marca el **inicio** y el **fin**; el **rectángulo**, una **acción** o proceso («mover 10 pasos»); el **rombo**, una **decisión** con una pregunta de respuesta sí/no («¿toca el borde?»), del que salen dos caminos; y el **paralelogramo**, una **entrada o salida** de datos («pedir el nombre», «mostrar resultado»). Si una flecha vuelve

hacia atrás, el diagrama tiene un **bucle**. Así se detectan errores de lógica antes de escribir el código.



1.3 La programación por bloques

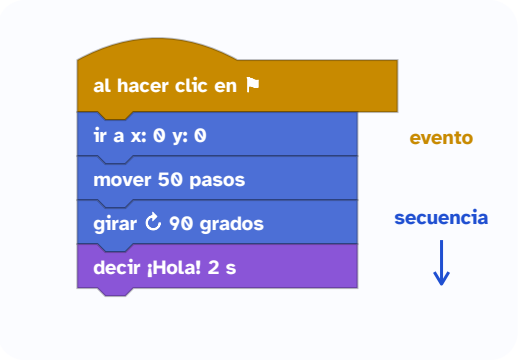
En la programación por bloques, las instrucciones son piezas de colores que se **encajan** como un puzle, en lugar de escribirse con texto. Así no hay errores de escritura y es fácil ver la estructura del programa. El entorno más conocido es **Scratch**, creado por el MIT; otros son **MakeCode** (para micro:bit), **mBlock** o **Arduinoblocks** (para robots), **App Inventor** (para aplicaciones de móvil) y **Snap!**. En Scratch hay un **escenario**, donde ocurre la acción; unos **objetos** o *sprites*, los personajes, cada uno con sus **disfraces** y sonidos; y para cada objeto, unos **programas** o *scripts* formados por bloques. Los bloques se agrupan por **categorías** de colores: movimiento, apariencia, sonido, eventos, control, sensores, operadores y variables.



2.1 Eventos y secuencias

Un programa por bloques empieza siempre con un evento: un bloque con forma de «sombrero» que indica **cuándo** se ejecuta el script. Por ejemplo: «al hacer clic en la bandera verde», «al presionar la tecla espacio», «al hacer clic en este objeto» o «al recibir un mensaje». Debajo del evento se encajan las instrucciones,

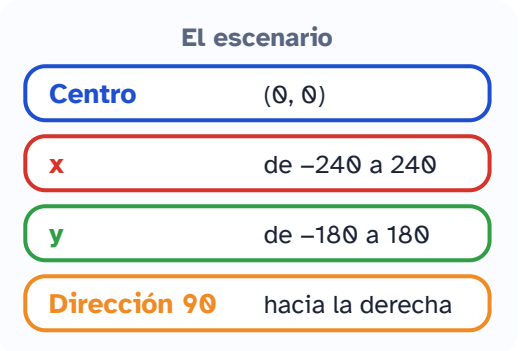
que se ejecutan en **secuencia**, de arriba abajo, una tras otra. El orden importa: no es lo mismo «girar 90° y mover 50 pasos» que «mover 50 pasos y girar 90°». Varios scripts pueden ejecutarse **a la vez** (en paralelo), cada uno con su propio evento, y los objetos se pueden comunicar entre sí **enviando mensajes**.



2.2 Coordenadas y movimiento en el escenario

En Scratch, el escenario es un rectángulo de 480 × 360 puntos con un sistema de **coordenadas cartesianas**: el centro es el punto (0, 0); la coordenada **x** va de −240 (izquierda) a 240 (derecha) y la **y** de −180 (abajo) a 180 (arriba). Con el bloque «ir a x: … y: …» un objeto salta a un punto exacto, y con «deslizar en 1 segundos a x: … y: …» se desplaza suavemente.

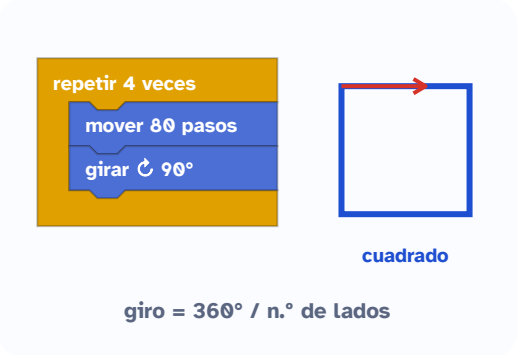
Para mover un objeto con el teclado se usan los bloques «cambiar x en 10» (a la derecha) o «cambiar x en −10» (a la izquierda), y lo mismo con y. La **dirección** se mide en grados: 90 es a la derecha, −90 a la izquierda, 0 hacia arriba y 180 hacia abajo; «mover 10 pasos» avanza en la dirección a la que apunta el objeto. Los bloques «rebotar si toca un borde» y «fijar estilo de rotación» evitan que los personajes salgan del escenario o aparezcan cabeza abajo.



2.3 Bucles: repetir instrucciones

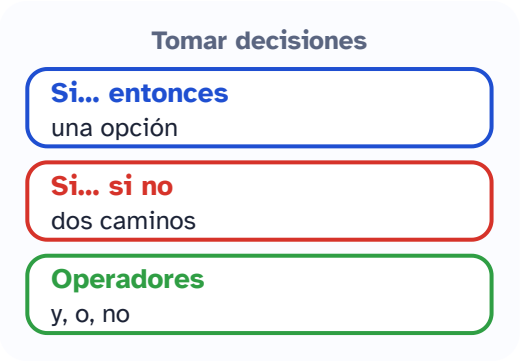
Un bucle repite un grupo de instrucciones varias veces, sin tener que copiarlas. Hay tres tipos principales. «**Repetir N veces**» ejecuta el bloque un número fijo de veces: para dibujar un cuadrado basta con «repetir 4: mover 100, girar 90°». «**Por siempre**» repite sin fin, mientras el programa esté en marcha: se usa para comprobar continuamente si se pulsa una tecla o si el personaje toca algo. «**Repetir hasta que**» repite mientras no se cumpla una condición: «repetir hasta que toque la meta». Los bucles hacen los programas más **cortos**, más **fáciles de leer** y de modificar. Para dibujar un polígono de n lados, el ángulo de giro es **360° / n**.

Los bucles se pueden **anidar**: meter un bucle dentro de otro. Por ejemplo, para dibujar una flor de 12 pétalos se repite 12 veces «dibujar un pétalo y girar 30°», y cada pétalo es, a su vez, un bucle que dibuja un arco. Hay que vigilar el número total de repeticiones: un bucle de 10 dentro de otro de 10 ejecuta 100 veces las instrucciones interiores. Para dibujar un polígono regular de n lados, el giro siempre es 360 / n grados, porque al final el lápiz debe haber dado una vuelta completa.



2.4 Condicionales: tomar decisiones

Un condicional permite que el programa **decida** qué hacer según se cumpla o no una condición. El bloque «**si … entonces**» ejecuta sus instrucciones solo si la condición es verdadera: «si toca el color rojo, entonces volver al inicio». El bloque «**si … entonces … si no**» tiene dos caminos: uno si se cumple y otro si no. Las condiciones se construyen con bloques de **sensores** (¿tocando el ratón?, ¿tecla presionada?, ¿tocando a otro objeto?) y de **operadores**: de comparación (**>, <, =**) y lógicos (**y**, **o**, **no**). Un condicional dentro de un bucle «por siempre» es la base de casi todos los videojuegos: comprueba sin parar lo que ocurre y reacciona.



2.5 Variables y operadores

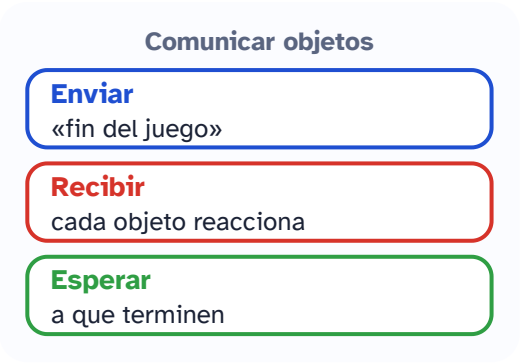
Una variable es un espacio de la memoria con un **nombre** donde el programa guarda un **dato que puede cambiar**: la puntuación, las vidas, el tiempo, el nombre del jugador. Con los bloques de variables se puede **crear** una variable, **darle un valor** («dar a puntos el valor 0»), **cambiarla** («sumar 1 a puntos») y **mostrarla** en el escenario. Los **operadores** realizan cálculos (suma, resta, multiplicación, división, número aleatorio entre 1 y 10), comparan valores y unen textos. Por ejemplo, para un juego de preguntas: «preguntar ¿cuánto es 7 × 8? y esperar»; «si respuesta = 56, entonces sumar 1 a puntos». Las **listas** guardan muchos datos en una sola variable.



2.6 Mensajes: comunicar objetos

En un programa con varios objetos, a menudo uno tiene que avisar a otro de que ha pasado algo: el jugador ha perdido, ha empezado un nivel nuevo, se ha pulsado un botón del menú. Para eso se usan los **mensajes**. Con el bloque «enviar mensaje [fin del juego]» un objeto emite un aviso, y todos los objetos que tengan el evento «al recibir [fin del juego]» ejecutan sus instrucciones: el escenario cambia de fondo, la música se detiene y aparece el texto «Game over».

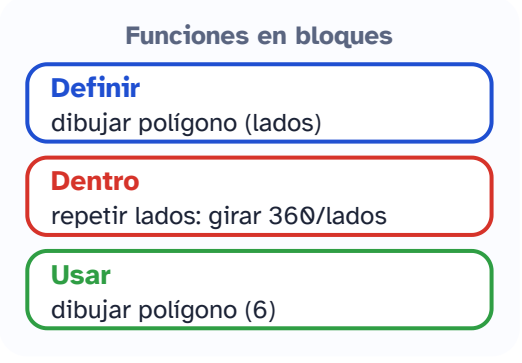
El bloque «enviar mensaje y esperar» hace que el objeto que lo envía espere a que los demás terminen de ejecutar sus scripts antes de continuar. Los mensajes permiten **organizar** el programa en partes que se comunican, en lugar de meter todo en un único script larguísimo, y facilitan crear menús, niveles y escenas como en una película.



2.7 Bloques propios: crear funciones

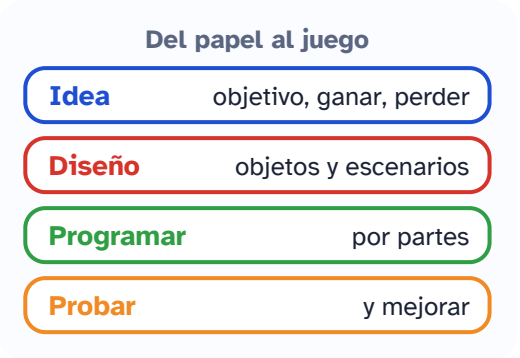
Cuando un grupo de instrucciones se repite en varios lugares del programa, se puede crear un **bloque propio** (en la categoría «Mis bloques»). Es lo que en otros lenguajes se llama **función** o procedimiento: se le da un nombre, como «dibujar cuadrado», se define una vez qué hace y después se usa como cualquier otro bloque.

Los bloques propios pueden tener **parámetros**: datos que se les pasan al usarlos. Por ejemplo, «dibujar polígono (lados) (tamaño)» puede dibujar un triángulo, un cuadrado o un hexágono según los valores que reciba: repite «lados» veces mover «tamaño» pasos y girar 360 / lados grados. Así el programa es más corto, más fácil de leer y de corregir: si hay un error, se arregla en un solo sitio. Dividir un problema grande en bloques propios es una forma de aplicar la **descomposición**, una de las bases del pensamiento computacional.



3.1 Diseñar un videojuego

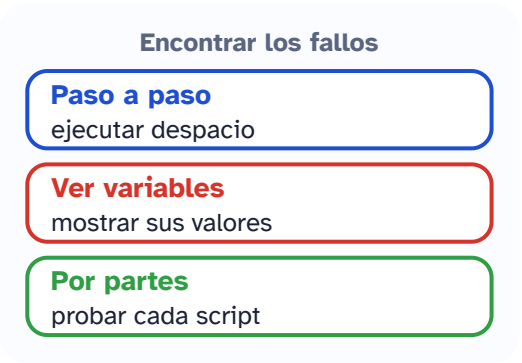
Para crear un programa más grande, como un **videojuego**, conviene seguir el proceso tecnológico. Primero, **definir** la idea: de qué trata, cuál es el objetivo, cómo se gana y cómo se pierde. Segundo, **descomponer** el problema en partes: el movimiento del personaje con las flechas, los enemigos que se mueven solos, la puntuación, las vidas, el fin del juego. Tercero, **programar** cada parte por separado y probarla. Cuarto, **unir** las partes y comunicar los objetos con mensajes. Por último, **probar** con otras personas y mejorar. Programar bien también es escribir programas ordenados: dar nombres claros a las variables y objetos y crear **bloques propios** (funciones) para las tareas que se repiten.



3.2 Errores y depuración

Es normal que un programa no funcione a la primera. Los fallos se llaman errores o *bugs*, y el proceso de encontrarlos y corregirlos, **depuración**. Hay errores de **secuencia** (instrucciones en el orden incorrecto), de **lógica** (una condición mal planteada, por ejemplo «>» en lugar de «<»), de **bucles infinitos** que nunca terminan y de **inicialización** (olvidar poner la puntuación a 0 al empezar). Estrategias para depurar: ejecutar el programa **paso a paso**; **mostrar las variables** en pantalla para ver cómo cambian; hacer que el personaje «diga» valores intermedios; probar cada parte por separado; y explicar el programa en voz alta a otra persona, que a menudo revela el fallo.

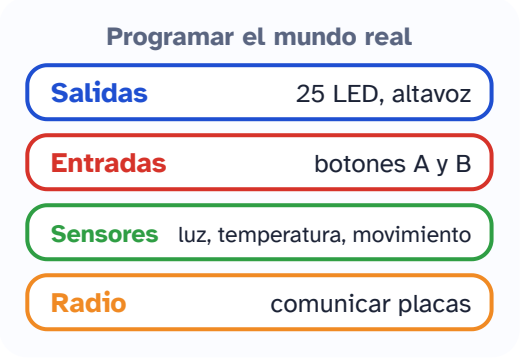
Algunas técnicas de depuración: ejecutar el programa **paso a paso**, mostrar en pantalla el valor de las variables mientras se ejecuta, probar cada parte por separado antes de unirla y explicar el programa en voz alta a otra persona (o a un patito de goma): al contarlo, a menudo descubrimos el fallo. Conviene guardar versiones del proyecto para poder volver atrás si algo deja de funcionar.



3.3 Programar una placa con bloques: micro:bit

La programación por bloques no sirve solo para la pantalla: también controla objetos reales. La placa **micro:bit**, creada por la BBC para la educación, se programa desde el navegador con **MakeCode**, con bloques muy parecidos a los de Scratch. Tiene una matriz de 25 LED, dos botones (A y B), sensor de luz, de temperatura, acelerómetro (detecta movimientos e inclinación), brújula, altavoz, micrófono y radio para comunicarse con otras placas.

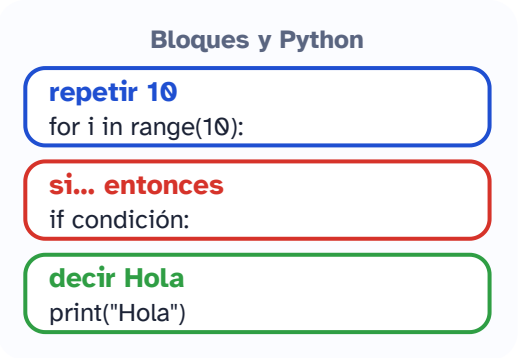
Un programa típico: «al presionar el botón A, mostrar icono corazón»; «al agitar, mostrar un número aleatorio del 1 al 6» (un dado electrónico); o «para siempre: si la temperatura > 25, mostrar flecha arriba». El editor incluye un **simulador** para probar el programa antes de descargarlo a la placa. Desde sus pines se pueden conectar LED, motores, servos y sensores externos. Arduino también se puede programar con bloques (mBlock, ArduinoBlocks) y Lego o VEX tienen sus propios entornos para robots.



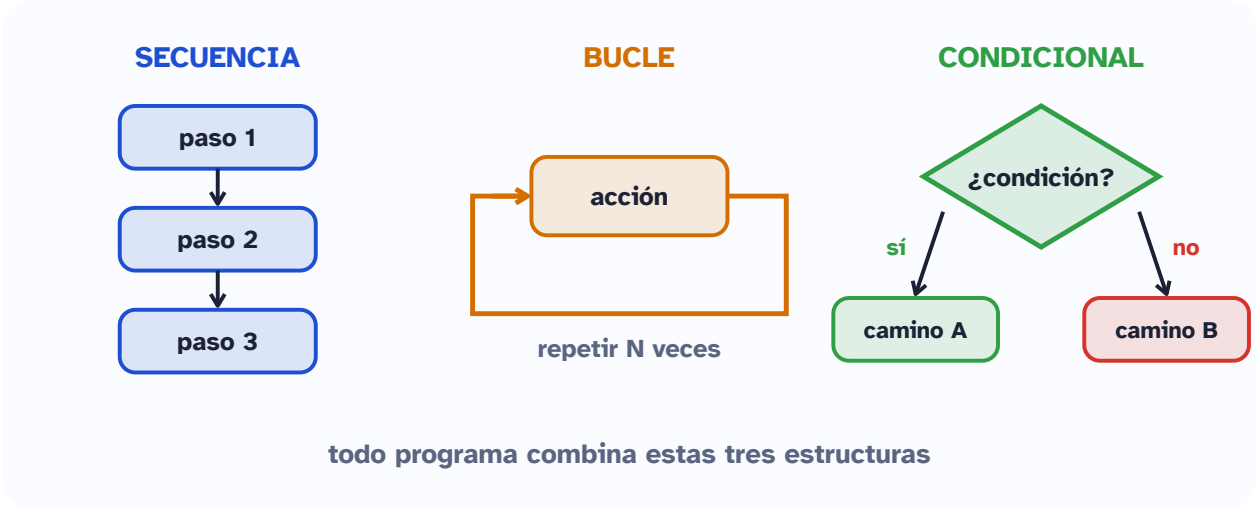
3.4 De los bloques al código

La programación por bloques es la puerta de entrada a los **lenguajes de texto**, como **Python**, **JavaScript** o **C++**, que usan las mismas ideas: secuencias, bucles, condicionales, variables y funciones. Muchos entornos, como MakeCode o mBlock,

permiten ver el **código equivalente** de un programa de bloques. Por ejemplo, «repetir 4 veces: mover 100, girar 90°» en Python sería un bucle `for i in range(4):`. La programación por bloques también sirve para controlar **robots** y placas como **micro:bit** o **Arduino**, que veremos en el tema de robótica, y para crear **aplicaciones de móvil** con App Inventor.



Las tres estructuras de control



Ejercicios

1. **Define algoritmo y pon un ejemplo de la vida cotidiana.**

2. **Dibuja los símbolos del diagrama de flujo para inicio, acción y decisión.**

3. **Nombra cuatro eventos que pueden iniciar un script.**

4. **¿Qué tres tipos de bucle hay en Scratch?**

5. **¿Qué ángulo hay que girar para dibujar un hexágono?**

6. **¿Qué diferencia hay entre «si … entonces» y «si … entonces … si no»?**

7. **¿Qué es una variable? Da tres ejemplos en un juego.**

8. **Escribe un programa que pregunte «¿cuánto es 6 × 7?» y diga «¡Bien!» o «Inténtalo otra vez».**

9. **Nombra tres tipos de errores de programación.**

10. **Cita dos entornos de programación por bloques y para qué sirven.**

11. **¿Qué coordenadas tiene el centro del escenario de Scratch? ¿Y la esquina inferior derecha?**

12. **¿Qué dirección hay que fijar para que un objeto mire hacia abajo?**

13. **¿Para qué sirven los mensajes en Scratch?**

14. **¿Qué es un bloque propio o función?**

15. **Nombra cuatro sensores o entradas de la placa micro:bit.**

16. **Escribe en pseudocódigo el algoritmo para preparar un vaso de cacao.**

17. **Escribe los bloques para mover un objeto con las cuatro flechas del teclado.**

18. **¿Qué diferencia hay entre «enviar mensaje» y «enviar mensaje y esperar»?**

19. **Diseña un bloque propio «saltar» para un personaje y explica sus instrucciones.**

20. **Programa en micro:bit un termómetro que muestre la temperatura al pulsar A y un aviso si supera 30 °C.**

21. **Un objeto sale del escenario y desaparece. ¿Qué bloques lo evitan?**

22. **¿Por qué los bloques propios hacen más fácil depurar un programa?**

23. **Explica tres técnicas de depuración.**

24. **Crea un menú de inicio con mensajes: un botón «Jugar» que al pulsarlo empiece el juego.**

25. **Traduce a Python: «repetir 5 veces: decir Hola».**

26. **Diseña un juego de atrapar objetos que caen: objetos, variables, eventos, mensajes y condición de fin.**

27. **Crea un bloque propio «estrella (puntas) (tamaño)» y explica qué ángulo de giro usarías para una estrella de 5 puntas.**

28. **Dos micro:bit se comunican por radio. Diseña un sistema de votación de clase: cada alumno pulsa A (sí) o B (no) y una placa central cuenta.**

29. **Un juego se congela al llegar a `0` vidas. Propón tres causas posibles y cómo comprobar cada una.**

30. **Compara Scratch, MakeCode y Python: qué se puede hacer con cada uno y por qué se aprende en ese orden.**

Practica

1. **Una secuencia ordenada y finita de instrucciones es un...**

   A) algoritmo
   B) sprite
   C) evento
   D) dibujo

2. **En un diagrama de flujo, la decisión se dibuja con un...**

   A) rectángulo
   B) rombo
   C) óvalo
   D) triángulo

3. **Scratch fue creado por el...**

   A) MIT
   B) CERN
   C) Google
   D) la NASA

4. **Los personajes de Scratch se llaman...**

   A) objetos (sprites)
   B) variables
   C) escenarios
   D) listas

5. **El bloque con forma de sombrero es un...**

   A) evento
   B) bucle
   C) operador
   D) condicional

6. **Para repetir siempre se usa...**

   A) repetir 10
   B) por siempre
   C) si ... entonces
   D) repetir hasta que

7. **Para dibujar un pentágono se gira...**

A) 72°
B) 60°
C) 90°
D) 45°

8. **Un dato que cambia durante el programa se guarda en una...**

A) variable
B) constante
C) disfraz
D) lista vacía

9. **El operador «y» exige que...**

A) se cumpla una condición
B) se cumplan las dos
C) no se cumpla ninguna
D) que no se cumpla

10. **Buscar y corregir errores se llama...**

A) compilar
B) depurar
C) ejecutar
D) decorar

11. **El centro del escenario de Scratch es...**

A) (240, 180)
B) (0, 0)
C) (–240, –180)
D) (100, 100)

12. **Para que un objeto mire a la izquierda, su dirección es...**

A) 90
B) –90
C) 0
D) 180

13. **«Cambiar y en 10» mueve el objeto...**

A) a la derecha
B) hacia arriba
C) hacia abajo

D) a la izquierda

14. **Para avisar a otros objetos se usa...**

A) una variable

B) un mensaje

C) un disfraz

D) un sonido

15. **Un bloque propio equivale en otros lenguajes a una...**

A) variable

B) función

C) lista

D) condición

16. **Un parámetro es...**

A) un error

B) un dato que recibe un bloque propio

C) un fondo

D) un sonido

17. **La placa micro:bit se programa con...**

A) MakeCode

B) Photoshop

C) Excel

D) Audacity

18. **El acelerómetro de la micro:bit detecta...**

A) la luz

B) el movimiento y la inclinación

C) el sonido

D) la humedad

19. **Escribir un algoritmo con frases sencillas antes de programar es...**

A) pseudocódigo

B) depurar

C) compilar

D) un sprite

20. **Explicar el programa en voz alta para encontrar fallos es una técnica de...**

A) diseño

B) depuración

C) animación

D) compilación

Explícalo con tus palabras

Incluye: Qué es un algoritmo y cómo se dibuja un diagrama de flujo; La diferencia entre un bucle y un condicional; Para qué sirve una variable en un juego.

Resumen



Glosario

Algoritmo	Secuencia ordenada y finita de instrucciones.
Programa	Algoritmo escrito en un lenguaje que entiende el ordenador.
Pseudocódigo	Algoritmo escrito con frases sencillas.
Diagrama de flujo	Representación gráfica de un algoritmo.
Evento	Suceso que inicia un script.
Coordenadas	Pareja (x, y) que indica una posición.
Bucle	Estructura que repite instrucciones.
Condicional	Estructura que elige según una condición.
Variable	Espacio con nombre que guarda un dato que cambia.
Operador	Símbolo para calcular o comparar.
Mensaje	Aviso entre objetos de un programa.
Bloque propio	Función definida por el programador.
Parámetro	Dato que recibe una función.
Depurar	Buscar y corregir errores.
micro:bit	Placa programable para aprender.
Simulador	Programa que imita la placa para probar sin montarla.

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Algoritmo

Secuencia ordenada y finita de instrucciones.

Diagrama de flujo

Óvalo, rectángulo, rombo y flechas.

Evento

Lo que hace empezar un script.

Bucle

Repite instrucciones: N veces, siempre o hasta que.

Condicional

Si … entonces … si no.

Variable

Dato con nombre que puede cambiar.

Polígono

Giro = 360° / n.º de lados.

Depurar

Encontrar y corregir errores.

Coordenadas en Scratch

x de –240 a 240; y de –180 a 180.

Mensaje

Aviso que un objeto envía y otros reciben.

Bloque propio

Función creada por el programador, con parámetros.

micro:bit

Placa educativa programable con bloques.

Soluciones

1. **A)** Algoritmo.
2. **B)** Rombo.
3. **A)** El MIT.
4. **A)** Objetos.
5. **A)** Evento.
6. **B)** Por siempre.
7. **A)** 360 / 5 = 72°.
8. **A)** Variable.
9. **B)** Las dos.
10. **B)** Depurar.
11. **B)** Origen de coordenadas.
12. **B)** –90 grados.
13. **B)** y aumenta hacia arriba.
14. **B)** Enviar y recibir mensajes.
15. **B)** Función o procedimiento.
16. **B)** Como «lados» en un polígono.
17. **A)** Editor de bloques de Microsoft.
18. **B)** Agitar, inclinar.
19. **A)** A medio camino del lenguaje natural.
20. **B)** El método del patito de goma.

Soluciones de los ejercicios

1. Secuencia ordenada y finita de instrucciones para resolver un problema; una receta o lavarse los dientes.
2. Óvalo, rectángulo y rombo.
3. Clic en la bandera, presionar una tecla, clic en el objeto, recibir un mensaje.
4. Repetir N veces, por siempre y repetir hasta que.
5. 360° / 6 = 60°.
6. El primero solo actúa si se cumple la condición; el segundo tiene también un camino para cuando no se cumple.
7. Un espacio de memoria con nombre que guarda un dato que cambia; puntos, vidas, tiempo.
8. Preguntar y esperar; si respuesta = 42 entonces decir «¡Bien!» si no decir «Inténtalo otra vez».
9. De secuencia, de lógica, bucles infinitos (o de inicialización).
10. Scratch (juegos y animaciones) y MakeCode (micro:bit) o App Inventor (apps de móvil).
11. Centro: (0, 0). Esquina inferior derecha: (240, –180).
12. 180 grados.
13. Para que un objeto avise a otros de que ha ocurrido algo y estos ejecuten sus instrucciones al recibir el mensaje.
14. Un bloque creado por el programador que agrupa instrucciones con un nombre y se puede usar muchas veces, con o sin parámetros.
15. Botones A y B, sensor de luz, sensor de temperatura, acelerómetro, brújula y micrófono.
16. 1) Coger un vaso. 2) Echar leche. 3) Añadir dos cucharadas de cacao. 4) Remover hasta que se disuelva. 5) Fin.
17. Al presionar flecha derecha: cambiar x en 10. Izquierda: cambiar x en –10. Arriba: cambiar y en 10. Abajo: cambiar y en –10.
18. El primero continúa enseguida; el segundo espera a que terminen todos los scripts que han recibido el mensaje.
19. Definir «saltar»: repetir 10 veces cambiar y en 10; repetir 10 veces cambiar y en –10. Se usa al presionar la barra espaciadora.
20. Al presionar A: mostrar número temperatura. Para siempre: si temperatura > 30, mostrar icono de aviso.
21. «Rebotar si toca un borde» dentro del bucle de movimiento, o comprobar con un condicional que x e y no superan los límites.
22. Porque el código repetido está en un solo sitio: si hay un error, se corrige una vez y se arregla en todos los lugares donde se usa.
23. Ejecutar paso a paso, mostrar el valor de las variables, probar cada parte por separado (y explicar el programa en voz alta).
24. Objeto botón: al hacer clic en este objeto, enviar mensaje «empezar» y esconder. Escenario: al recibir «empezar», cambiar fondo. Personajes: al recibir «empezar», mostrar e iniciar sus scripts.

25. for i in range(5): print("Hola")
26. Objetos: cesta (se mueve con flechas) y fruta (cae desde una x aleatoria). Variables: puntos, vidas. Si la fruta toca la cesta: puntos +1 y vuelve arriba. Si toca el suelo: vidas −1. Si vidas = 0: enviar «fin del juego»; al recibirlo, todos se detienen y aparece la puntuación.
27. Repetir puntas veces: mover tamaño pasos y girar 720 / puntas grados. Para 5 puntas, 144°: así el lápiz da dos vueltas completas y forma la estrella.
28. Placas de alumnos: al presionar A, enviar por radio 1; al presionar B, enviar 0. Placa central: al recibir número, si es 1, sí +1; si es 0, no +1; al presionar A+B, mostrar resultados. Todas en el mismo grupo de radio.
29. 1) Un «repetir hasta que» cuya condición nunca se cumple: mostrar la variable vidas. 2) Un «enviar y esperar» cuyo receptor nunca termina: revisar sus scripts. 3) Un «detener todo» mal colocado: ejecutar paso a paso.
30. Scratch: juegos y animaciones en pantalla. MakeCode: programar placas y objetos reales. Python: lenguaje de texto profesional (datos, IA, web). Se empieza por bloques para aprender las estructuras sin errores de escritura y después se pasa a texto, donde las mismas ideas se escriben con palabras.